

CAHIER DES CHARGES FONCTIONNEL

EDLoc

Application de gestion des états des lieux d'entrée et de sortie

Projet de portfolio — Concepteur Développeur d'Applications

Auteur : Inès SAMEL

Version 1.0 — Juillet 2026

Sommaire

1. Présentation du projet

1.1 Contexte

Lors de la mise en location d'un bien immobilier, le bailleur et le locataire doivent réaliser un état des lieux d'entrée, puis un état des lieux de sortie au terme du bail. Ce document contractuel, encadré par la loi du 6 juillet 1989, sert de référence pour comparer l'état du logement entre le début et la fin de la location, et pour déterminer les éventuelles retenues sur le dépôt de garantie.

En pratique, ces états des lieux sont encore très souvent réalisés sur papier : ils sont chronophages, difficiles à archiver, peu précis (absence de preuve photo horodatée) et sources de litiges lorsque les deux parties ne sont pas d'accord sur l'état constaté.

EDLoc propose de dématérialiser entièrement ce processus grâce à une application mobile/tablette permettant de réaliser un état des lieux collaboratif, illustré par des photos horodatées, signé électroniquement par les deux parties et exportable en PDF.

1.2 Objectifs du projet

- Simplifier et accélérer la réalisation des états des lieux pour les particuliers (sans passer par une agence).
- Sécuriser juridiquement la relation bailleur/locataire grâce à des preuves horodatées et une signature électronique.
- Faciliter la comparaison entre l'état d'entrée et l'état de sortie pour objectiver les dégradations éventuelles.
- Centraliser l'historique des états des lieux d'un bien (utile en cas de location successive).

1.3 Enjeux

- Enjeu juridique : produire un document conforme et opposable en cas de litige.
- Enjeu d'usage : une application utilisable par tout public (pas seulement les utilisateurs à l'aise avec le numérique), sur le lieu même de la visite, depuis un mobile ou une tablette (connexion internet requise).
- Enjeu de confiance : garantir que les deux parties valident le même contenu (pas de modification après signature).

2. Acteurs du projet

Acteur	Rôle dans l'application
Bailleur (propriétaire)	Crée l'état des lieux, saisit les pièces et équipements (ses observations et celles du locataire relayées à l'oral), ajoute des photos, signe le document, génère et partage le PDF final.
Locataire	Présent physiquement aux côtés du bailleur lors de la visite, formule à l'oral ses observations sur chaque pièce (relayées et saisies par le bailleur dans l'application), consulte le récapitulatif final et signe électroniquement pour valider son accord, sur le même appareil que le bailleur.
Administrateur (technique)	Supervise la plateforme, gère les comptes en cas de litige technique, assure la maintenance (acteur système, hors interface grand public).

Remarque de cadrage : l'application est utilisée sur un seul appareil (celui du bailleur). Le locataire n'a pas de compte ni d'accès autonome ; il participe physiquement à la visite et n'interagit directement avec l'application qu'au moment de la relecture et de la signature finale, sur l'appareil du bailleur. L'ajout d'un rôle « agence / mandataire » est identifié comme une évolution possible (voir 3.2).

3. Périmètre du projet

3.1 Fonctionnalités incluses (version 1)

- Création d'un compte et connexion (bailleur uniquement).
- Création d'un état des lieux d'entrée, pièce par pièce et élément par élément.
- Saisie de l'état des lieux par le bailleur, sur un seul appareil, en présence du locataire qui formule ses observations à l'oral pendant la visite du logement.
- Ajout de photos horodatées, associées à chaque pièce ou élément.
- Signature électronique du bailleur et du locataire.
- Génération automatique d'un PDF récapitulatif, envoyé ou téléchargeable par les deux parties.
- Création d'un état des lieux de sortie à partir de l'état des lieux d'entrée du même bien.
- Comparaison automatique entre l'état d'entrée et l'état de sortie, avec mise en évidence des différences constatées (nouvelles remarques, nouvelles photos, changement de statut d'un élément).
- Historique des états des lieux réalisés, consultables par le bailleur.

3.2 Hors périmètre (évolutions futures)

- Gestion complète du bail (loyers, quittances, révision de loyer).
- Messagerie instantanée intégrée entre bailleur et locataire.
- Rôle « agence immobilière » multi-biens et multi-mandats.
- Chiffrage automatique des retenues sur dépôt de garantie.
- Signature électronique certifiée eIDAS niveau qualifié (v1 : signature simple avec valeur probatoire standard).
- Mode hors-ligne : saisie de l'état des lieux sans connexion, avec synchronisation automatique au retour du réseau.
- Monétisation du service : deux modèles sont envisagés — un abonnement mensuel ou une facturation à l'état des lieux — avec une intégration Stripe côté API. Le modèle freemium a été écarté en raison du risque de contournement par création de comptes multiples. Cette évolution est conditionnée à la création d'une structure juridique (micro-entreprise) et à la rédaction de CGV ; l'architecture retenue (API REST séparée) permet de l'ajouter sans refonte.
- Reprise d'un état des lieux d'entrée antérieur à l'application (changement de bailleur en cours de bail, état des lieux papier existant) : saisie a posteriori d'une entrée déjà signée, permettant de réaliser la sortie et la comparaison dans l'application. En v1, la création d'un état des lieux de sortie exige une entrée signée dans l'application (RG14).

4. Description fonctionnelle détaillée

4.1 Création et remplissage d'un état des lieux

Le bailleur crée un nouvel état des lieux en renseignant les informations du bien (adresse, type de logement, nombre de pièces) et le type d'état des lieux (entrée ou sortie). La visite se déroule ensuite avec le locataire physiquement présent : les deux parties font le tour du logement ensemble, et le bailleur saisit dans l'application, pièce par pièce, ses propres observations ainsi que celles formulées à l'oral par le locataire.

L'état des lieux est structuré en pièces (ex. : cuisine, salon, chambre, salle de bain), chacune contenant une liste d'éléments prédéfinis mais modifiables (murs, sol, plafond, fenêtres, équipements électroménagers, robinetterie, etc.). Pour chaque élément, les deux parties renseignent un état (neuf, bon état, état d'usage, mauvais état) et peuvent ajouter un commentaire libre.

4.2 Ajout de photos horodatées

Chaque pièce et chaque élément peut être illustré par une ou plusieurs photos, prises directement depuis l'application. Chaque photo est automatiquement horodatée (date et heure) et associée à l'élément concerné, afin de constituer une preuve fiable de l'état constaté au moment de la visite.

4.3 Signature électronique

Une fois l'ensemble des pièces renseignées, le document est présenté en relecture aux deux parties. Chacune signe électroniquement (signature manuscrite tactile ou code de validation envoyé par e-mail/SMS) directement dans l'application. Le document est alors verrouillé : plus aucune modification n'est possible après la double signature.

4.4 Génération et partage du PDF

Après signature, l'application génère automatiquement un PDF reprenant l'ensemble des informations saisies, les photos et les deux signatures. Ce PDF est envoyé par e-mail aux deux parties et reste consultable depuis l'application.

4.5 Comparaison entrée / sortie

Lors de la création d'un état des lieux de sortie, l'application propose de le générer à partir de l'état des lieux d'entrée existant pour le même bien. Elle affiche alors automatiquement, pour chaque élément, l'état constaté à l'entrée en vis-à-vis de l'état constaté à la sortie, et signale les éléments dont l'état a changé, afin de faciliter l'identification des dégradations éventuelles.

4.6 Administration de la plateforme

Un rôle administrateur, distinct du rôle bailleur, est réservé au développeur de la plateforme. Ce compte unique est créé directement en base de données (aucune inscription publique ne permet d'obtenir ce rôle) et accède à une interface d'administration minimale, disponible uniquement au format desktop : liste des bailleurs inscrits, désactivation / réactivation d'un compte, et suppression d'un compte. Toute suppression est précédée d'une confirmation et entraîne l'effacement des données associées (biens, états des lieux, photos), en cohérence avec le droit à la suppression prévu par le RGPD (voir section 5). Les routes d'administration de l'API sont protégées par une vérification du rôle porté par le jeton d'authentification.

5. Exigences non fonctionnelles

Catégorie	Exigence
Ergonomie	Interface pensée « mobile-first » / tactile, utilisable sans formation, adaptée à une utilisation debout, dans un logement, en conditions de luminosité variables.
Sécurité & RGPD	Données personnelles et photos stockées de façon sécurisée et chiffrée ; mots de passe stockés hachés avec Argon2id, jamais en clair ; consentement explicite des utilisateurs ; droit à l'export et à la suppression des données.
Disponibilité	L'application nécessite une connexion internet (Wi-Fi / 4G). Une perte de réseau est détectée et signalée à l'utilisateur afin d'éviter toute perte de saisie.

Catégorie	Exigence
Performance	Chargement d'une pièce et de ses photos en moins de 2 secondes en conditions réseau standard (4G).
Fiabilité juridique	Horodatage fiable des photos et de la signature ; document non modifiable après signature des deux parties.
Compatibilité	Application accessible sur smartphone et tablette (iOS et Android), et en version web responsive.

6. Règles de gestion

Les règles de gestion ci-dessous formalisent les contraintes métier que le système doit respecter. Elles sont numérotées (RGn) afin d'être référencées depuis les critères d'acceptation des user stories et depuis la conception (MCD/MLD/MPD).

Comptes et acteurs

- **RG1** — Seul le bailleur dispose d'un compte ; le locataire n'a pas d'accès autonome à l'application.
- **RG2** — L'adresse e-mail d'un bailleur est unique ; celle d'un locataire, si elle est renseignée, est unique.
- **RG3** — Le mot de passe du bailleur est stocké haché (Argon2id) et n'est jamais conservé en clair.

Biens et états des lieux

- **RG4** — Un bien appartient à un seul bailleur ; un bailleur peut posséder plusieurs biens.
- **RG5** — Un état des lieux porte sur un seul bien et concerne un seul locataire.
- **RG6** — Un état des lieux est de type « entrée » ou « sortie ».
- **RG7** — Un état des lieux contient au moins une pièce ; une pièce contient au moins un élément.
- **RG8** — Chaque élément a un état parmi : neuf, bon état, état d'usage, mauvais état.

Photos

- **RG9** — Une photo est rattachée à un élément et est horodatée automatiquement (date et heure) au moment de la prise.

Signature et verrouillage

- **RG10** — Un état des lieux ne peut être soumis à la signature que si toutes ses pièces et éléments prévus sont renseignés.
- **RG11** — Un état des lieux requiert deux signatures (bailleur et locataire) ; il ne peut exister qu'au plus une signature par rôle et par état des lieux.
- **RG12** — Après la double signature, l'état des lieux passe au statut « signé » et devient non modifiable (verrouillé).
- **RG13** — Le PDF récapitulatif est généré automatiquement dès la double signature, puis envoyé par e-mail aux deux parties.

Sortie et comparaison

- **RG14** — Un état des lieux de sortie est créé à partir du dernier état des lieux d'entrée signé du même bien et du même locataire.
- **RG15** — La comparaison met en évidence, pour chaque élément, tout changement d'état entre l'entrée et la sortie.
- **RG16** — La suppression d'un état des lieux entraîne la suppression en cascade de ses pièces, éléments, photos et signatures.

- **RG17** — Un compte bailleur doit être confirmé avant sa première connexion : à l'inscription, un e-mail de confirmation est envoyé et la connexion est refusée tant que le lien n'a pas été utilisé.
- **RG18** — Les jetons transmis par e-mail (confirmation de compte : 24 h ; réinitialisation de mot de passe : 1 h) sont à usage unique et stockés hachés en base.

7. Architecture et choix techniques

EDLoc repose sur une architecture client-serveur à trois niveaux : une application web responsive (utilisable sur mobile, tablette et ordinateur via un navigateur), une API REST indépendante qui centralise la logique métier, et une couche de données (base relationnelle et stockage de fichiers). Un principe directeur guide ces choix : un écosystème TypeScript unifié du front au back, afin de partager les types de données entre les couches et de limiter les erreurs d'intégration.

7.1 Vue d'ensemble

L'application cliente communique avec l'API via des requêtes HTTPS au format JSON. L'API applique les règles métier, accède aux données par l'intermédiaire de l'ORM, et délègue le stockage des fichiers volumineux (photos, PDF) à un service dédié. La base de données ne contient que les données structurées ; les fichiers sont référencés par leur URL. La version 1 nécessite une connexion internet ; le mode hors-ligne est prévu en évolution future (voir 3.2).

7.2 Choix technologiques

Couche	Technologie retenue	Justification	Alternatives
Application cliente	Next.js (React + TypeScript)	Web responsive, utilisable depuis un navigateur mobile/tablette/PC ; aucun store ni compte développeur payant ; accès à la caméra (getUserMedia) et à la signature tactile (canvas) ; déploiement gratuit et simple.	React Native / Flutter (comptes stores payants, appareils de test requis).
Interface (UI)	Tailwind CSS + shadcn/ui	Composants copiés dans le projet et entièrement personnalisables via des tokens (couleurs, rayons, typographie) : la charte EDLoc (crème, terracotta, boutons pilule, Nunito Sans) s'applique directement. shadcn/ui repose sur les primitives Radix UI, accessibles nativement (navigation clavier, focus visible, ARIA), en cohérence avec les engagements d'accessibilité de la charte.	MUI / Bootstrap (thèmes imposés, plus lourds à personnaliser) ; CSS Modules seuls.
Back-end / API	Node.js + Express	API REST indépendante ; séparation claire front/back ; TypeScript partagé avec le front ; écosystème mature.	API intégrée à Next.js (écartée pour montrer une API séparée) ; NestJS (plus lourd en solo).
ORM	Prisma	Accès aux données typé et sûr ; migrations versionnées ; schéma reflétant le MPD ; types partagés avec le code.	SQL natif (verbeux) ; TypeORM / Sequelize.
Validation des données	Zod	Validation des schémas de données à l'entrée de l'API (corps de requêtes, paramètres) avant tout traitement ; schémas TypeScript-first dont les types sont inférés et partagés avec le front	Yup / Joi (non TypeScript-first) ; class-validator.

Couche	Technologie retenue	Justification	Alternatives
		— en cohérence avec la stack TypeScript de bout en bout ; messages d'erreur exploitables côté interface.	
Base de données	PostgreSQL	Base relationnelle robuste ; intégrité forte (clés étrangères, ENUM, UUID).	MySQL / MariaDB.
Stockage fichiers	Stockage objet (S3-compatible)	Photos et PDF stockés hors base ; URL sécurisées ; scalable.	Stockage sur le serveur de fichiers (moins scalable).
Authentification	JWT (mot de passe haché avec Argon2id)	Authentification sans état, adaptée à une API REST mobile. Argon2id (lauréat de la Password Hashing Competition, recommandé par l'OWASP) est préféré à bcrypt car il est « memory-hard » : très résistant aux attaques par force brute sur GPU/ASIC, avec des paramètres réglables (mémoire, itérations, parallélisme).	bcrypt (sûr et éprouvé, mais moins résistant aux attaques parallélisées).
Génération PDF	Bibliothèque serveur (PDFKit / Puppeteer)	PDF généré côté serveur à partir des données de l'EDL : rendu homogène et non modifiable.	Génération côté client (moins fiable).
Hébergement	Railway (front + API + PostgreSQL)	Déploiement simple sur une plateforme unique, compte déjà en place ; base PostgreSQL managée ; lien de démo public pour le portfolio.	—

7.3 Cohérence de la stack

L'usage de TypeScript de bout en bout (Next.js, Express, Prisma) permet de partager les types de données entre le front et le back, de réduire les erreurs d'intégration et d'accélérer un développement mené en solo.

Note sur Prisma et le modèle de données : le schéma Prisma (schema.prisma) est la traduction directe du MPD — chaque table devient un modèle, chaque clé étrangère une relation, les identifiants UUID se déclarent via `@default(uuid())` et les types ENUM PostgreSQL correspondent à des enum Prisma. Le modèle conceptuel et logique (MCD/MLD) reste donc inchangé.

8. Livrables attendus du projet

Livrable	Description
Cahier des charges fonctionnel	Le présent document.
User stories	Description des besoins utilisateurs au format « En tant que... je veux... afin de... ».
Diagrammes de cas d'utilisation (UML)	Représentation des interactions entre acteurs et fonctionnalités.
MCD	Modèle Conceptuel de Données (méthode Merise).
MLD	Modèle Logique de Données.
MPD	Modèle Physique de Données (script SQL de création des tables).
Maquettes / wireframes	Écrans principaux de l'application (mobile et web).
Application fonctionnelle	Version développée de l'application (front + back).

9. Planning prévisionnel (méthode agile)

Le projet sera mené selon une approche agile, en itérations courtes, adaptée à un développement en solo pour un portfolio.

Phase	Contenu
Phase 1 — Conception	Cahier des charges, user stories, cas d'utilisation, MCD/MLD/MPD.
Phase 2 — Maquettage	Wireframes et maquettes des écrans clés (parcours bailleur et locataire).
Phase 3 — Développement du back-end	Modèle de données, API, authentification, génération PDF.
Phase 4 — Développement du front	Écrans mobile/web, saisie collaborative, photos, signature.
Phase 5 — Tests & recette	Tests fonctionnels, corrections, jeu de données de démonstration.
Phase 6 — Présentation portfolio	Mise en valeur du projet (démonstration, documentation, dépôt de code).

10. Glossaire

Terme	Définition
État des lieux	Document décrivant l'état d'un logement à un instant donné (entrée ou sortie du locataire).
Bailleur	Propriétaire qui met un bien en location.
Locataire	Personne qui loue le bien.
MCD / MLD / MPD	Modèles conceptuel, logique et physique de données (méthode Merise).
Horodatage	Association automatique d'une date et d'une heure à une donnée (ici, une photo).
Argon2id	Algorithme de hachage de mots de passe (lauréat de la Password Hashing Competition, recommandé par l'OWASP), résistant aux attaques par force brute grâce à sa forte consommation mémoire.